Final Project Methods Report: Contextual Bandit Methods for Personalized Recommendation

Authors: Edwin Espinoza, Lucas Venetoulias
Course: DSC 291, Sequential Decision-Making (Ma)
Date: Feb. 15th, 2026

1. Dataset and Preprocessing

Our project makes use of the MovieLens 10M dataset (found here), released by GroupLens Research at the University of Minnesota. The dataset contains just over 10 million ratings from 71,567 users of the online movie recommendation service MovieLens on 10,681 movies. Each record consists of an user identifier, a movie identifier, a rating (on a scale from 1 to 5), and a timestamp.

To cast recommendations as a contextual bandits problem, we convert our ratings into binary rewards. Specifically, we choose the rewards to be

$$r_t = \begin{cases} 1 & \text{if rating} \geq 4, \\ 0 & \text{otherwise.} \end{cases} \tag{1.1}$$

This binary scale treats highly rated movies as successful recommendations, which is what we are looking to compare across the various algorithms of choice. (Note: we removed users whose interaction histories were sparse to reduce any instability in how our context vectors get created. See Section 2 below.)

For each interaction at a time step t, we construct a context vector $x_t \in \mathbb{R}^d$ using the available demographic information along with simple interaction statistics (historical average rating, historical average rating count, etc.). We normalize all continuous features such that they have mean zero and variance one.

2. Contextual Bandit Formulation

We are trying to model personalized recommendations as a contextual multi-armed bandits problem. At each time step $t \in \{1, 2, \ldots, T\}$ for some time horizon T, we have:

- A context vector $x_t \in \mathbb{R}^d$ is observed by the agent.
- A finite set of candidate items (arms) $\mathcal{A}_t \subseteq \mathcal{A}$ from all arms $\mathcal{A}$ is available for selection.
- An arm $a_t \in \mathcal{A}_t$ is selected by the agent.
- A reward $r_t \in \{0, 1\}$ is observed based on that arm.

Our goal is to maximize the cumulative reward over all T interactions. The performance of the agent is evaluated using cumulative regret, given by

$$R_T = \sum_{t=1}^{T} (r_t^* - r_t), \tag{2.1}$$

where r_t^* denotes the reward of the optimal action at time t.

3. Algorithms

We implement and compare the performance of three contextual multi-armed bandits algorithms, each of which makes use different exploration strategies.

3.1 Linear Upper Confidence Bound (LinUCB)

LinUCB assumes that the expected rewards are linear in the context features, or

$$\mathbb{E}[r_t \mid x_{t,a}] = x_{t,a}^\top \boldsymbol{\theta}, \tag{3.1.1}$$

for the feature vector $x_{t,a}$ associated with arm a at time t for the unknown parameter vector $\boldsymbol{\theta} \in \mathbb{R}^d$. We estimate the vector $\boldsymbol{\theta}$ using regularized least squares

$$\widehat{\boldsymbol{\theta}}_t = V_t^{-1} \sum_{s=1}^{t-1} x_{s,a_s} r_s \quad \text{for the design matrix } V_t = \sum_{s=1}^{t-1} x_{s,a_s} x_{s,a_s}^\top + \lambda I. \tag{3.1.2}$$

Establishing the standard ellipsoidal confidence regions around the estimate as

$$\mathcal{C}_t = \left\{ \boldsymbol{\theta} \mid (\boldsymbol{\theta} - \widehat{\boldsymbol{\theta}}_t)^\top V_t (\boldsymbol{\theta} - \widehat{\boldsymbol{\theta}}_t) \leq \sigma^2 \beta_t \right\} \tag{3.1.3}$$

where $\sigma^2 \beta_t$ controls for the radius of the confidence level, the LinUCB chooses the action that maximizes the highest possible reward over $\mathcal{C}_t$:

$$a_t = \arg\max_{a \in \mathcal{A}_t} \left\{ \max_{\boldsymbol{\theta} \in \mathcal{C}_t} x_{t,a}^\top \boldsymbol{\theta} \right\}. \tag{3.1.4}$$

Solving this inner maximization yields the following closed-form upper confidence bound:

$$a_t = \arg\max_{a \in \mathcal{A}_t} \left\{ x_{t,a}^\top \widehat{\boldsymbol{\theta}}_t + \sigma \sqrt{\beta_t} \sqrt{x_{t,a}^\top V_t^{-1} x_{t,a}} \right\}. \tag{3.1.5}$$

In this confidence bound, the first term represents the estimated expected reward, while the second quantifies uncertainty in the estimate $\widehat{\boldsymbol{\theta}}_t$ in the direction $x_{t,a}$. (That is, actions with higher uncertainty receive a larger exploration bonus, encouraging sampling in poorly explored regions of the feature space.)

3.2 Contextual Thompson Sampling

While LinUCB implements an exploration phase through deterministic optimism, Contextual Thompson Sampling adopts a Bayesian approach to estimating uncertainty. The reward model remains the same, namely $r_t = x_{t,a_t}^\top \boldsymbol{\theta} + \xi_t$ for some white noise process $\xi_t \sim \mathrm{WN}(0, \sigma^2)$. We assign a Gaussian prior over the parameter, namely $\boldsymbol{\theta} \sim \mathcal{N}(0, \lambda^{-1} I)$. If

$$\mathcal{D}_{t-1} = \left\{ (x_{1,a_1}, r_1), \ldots, (x_{t-1,a_{t-1}}, r_{t-1}) \right\} \tag{3.2.1}$$

denotes the observed data at time $t-1$, then Bayes' rule yields a Gaussian posterior, namely

$$\tilde{\boldsymbol{\theta}}_t = \boldsymbol{\theta} \mid \mathcal{D}_{t-1} \sim \mathcal{N}\left(\widehat{\boldsymbol{\theta}}_t, \sigma^2 V_t^{-1} \right) \tag{3.2.2}$$

from the estimate $\widehat{\boldsymbol{\theta}}_t$ and design matrix V_t given in equation (3.1.2). At each time step, Thompson sampling proceeds by sampling a parameter vector as described in equation (3.2.2) and chooses the action

$$a_t = \arg\max_{a \in \mathcal{A}_t} x_{t,a}^\top \tilde{\boldsymbol{\theta}}_t. \tag{3.2.3}$$

The exploration here emerges naturally from posterior uncertainty: scarce data leads to higher posterior variance and, thus, more diverse sampled parameters, while the posterior distribution concentrates as the observations accumulate.

3.3 ε-Greedy (Contextual)

As a baseline, we also consider a contextual ε-greedy algorithm under the same linear reward model, whose expectation is given in equation (3.1.1). If $\widehat{\boldsymbol{\theta}}_t$ denotes the ridge regression estimate computed from $\mathcal{D}_{t-1}$ (where $\mathcal{D}_{t-1}$ is defined in equation (3.2.1) above), then the contextual ε-greedy policy selects actions according to the rule:

- With probability $(1-\varepsilon)$, choose the greedy action

$$a_t = \arg\max_{a \in \mathcal{A}_t} x_{t,a}^\top \widehat{\boldsymbol{\theta}}_t. \tag{3.3.1}$$

- With probability ε, select an action uniformly at random from the available action set $\mathcal{A}_t$ at this time step.

The exploration rate $\varepsilon \in (0,1)$ controls for the trade-off between exploration and exploitation, with the exploration phase taking into account some randomization through the uniform selection of arms. In doing this, the algorithm does not adapt its exploration to the geometry of the feature space, but we use it as a useful benchmark to compare against.

4. Evaluation

4.1 Offline Evaluation via Replay

Since we could not use live deployment for our project, we adopt an offline replay evaluation protocol from the logged interaction data. For each logged event $E_t = (x_t, a_t^{\log}, r_t)$, a candidate algorithm selects an action a_t given x_t. If $a_t = a_t^{\log}$, then the reward r_t is revealed and the model is updated accordingly; otherwise, the interaction is skipped entirely. This way, we ensure fair comparisons across all algorithms under the assumption that the logging policy provides sufficient coverage of the action space. (We evaluate all of the candidate algorithms on identical logged interaction sequences.)

4.2 Evaluation Metrics

Algorithms were compared using the following metrics:

- Cumulative regret R_T.
- Average reward over time.
- Convergence behavior as reflected in learning curves.

To account for stochasticity in exploration, we repeat each experiment across a few random seeds, and we report the mean performance along with its corresponding standard deviation.